

Agosto 28, 2026

Patrones de Software

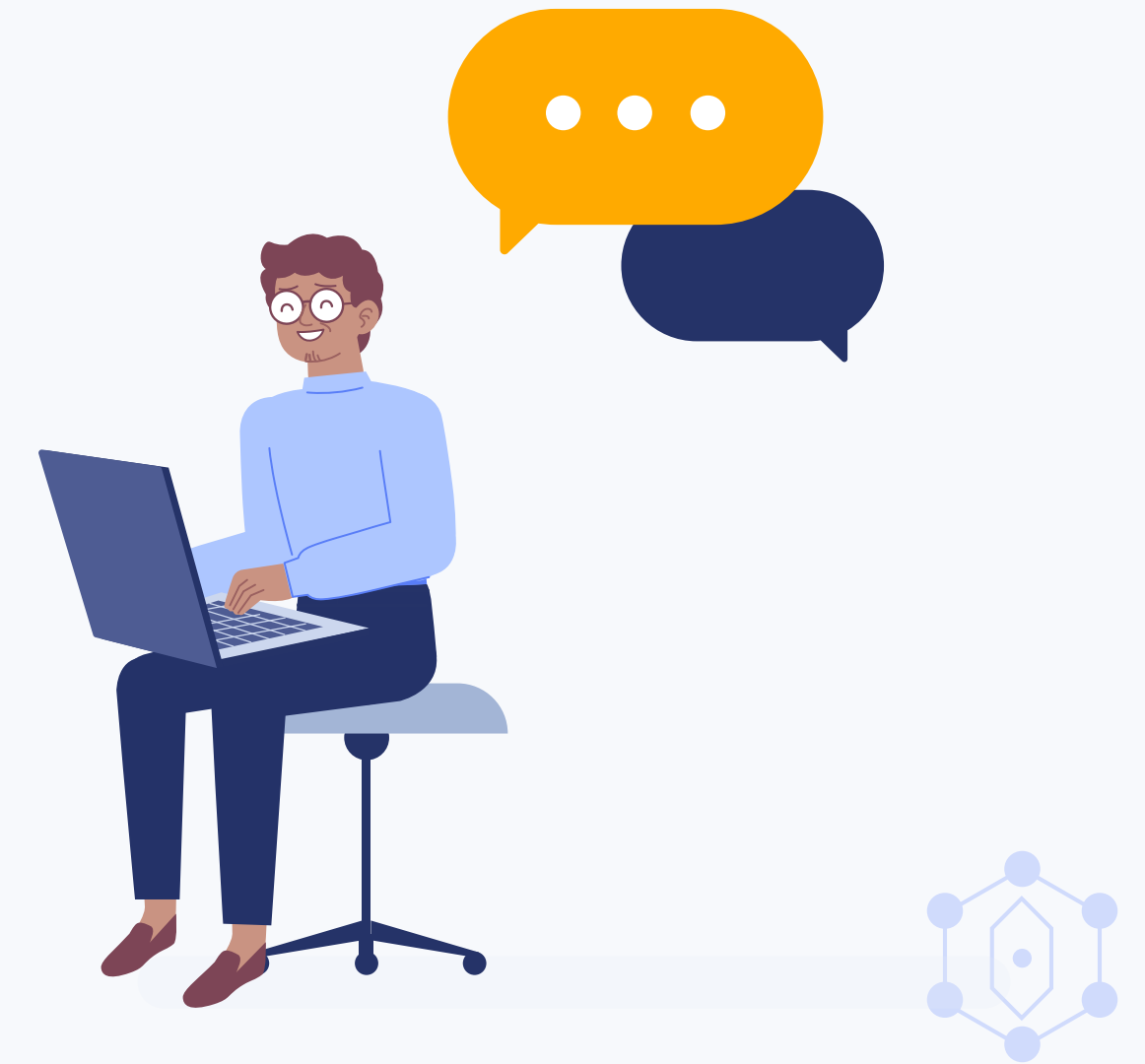
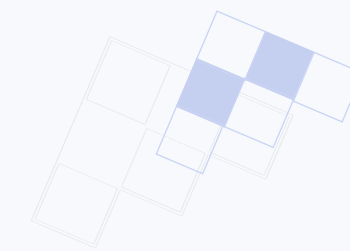
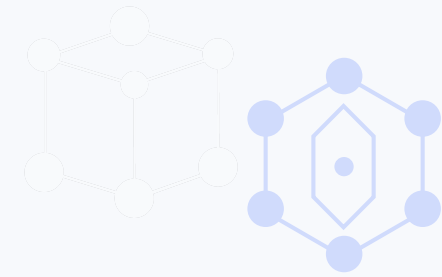
Docente: Jhonatan Andres Mideros Narvaez

Semestre: Cuarto



Recordemos

Conceptos vistos en la anterior clase

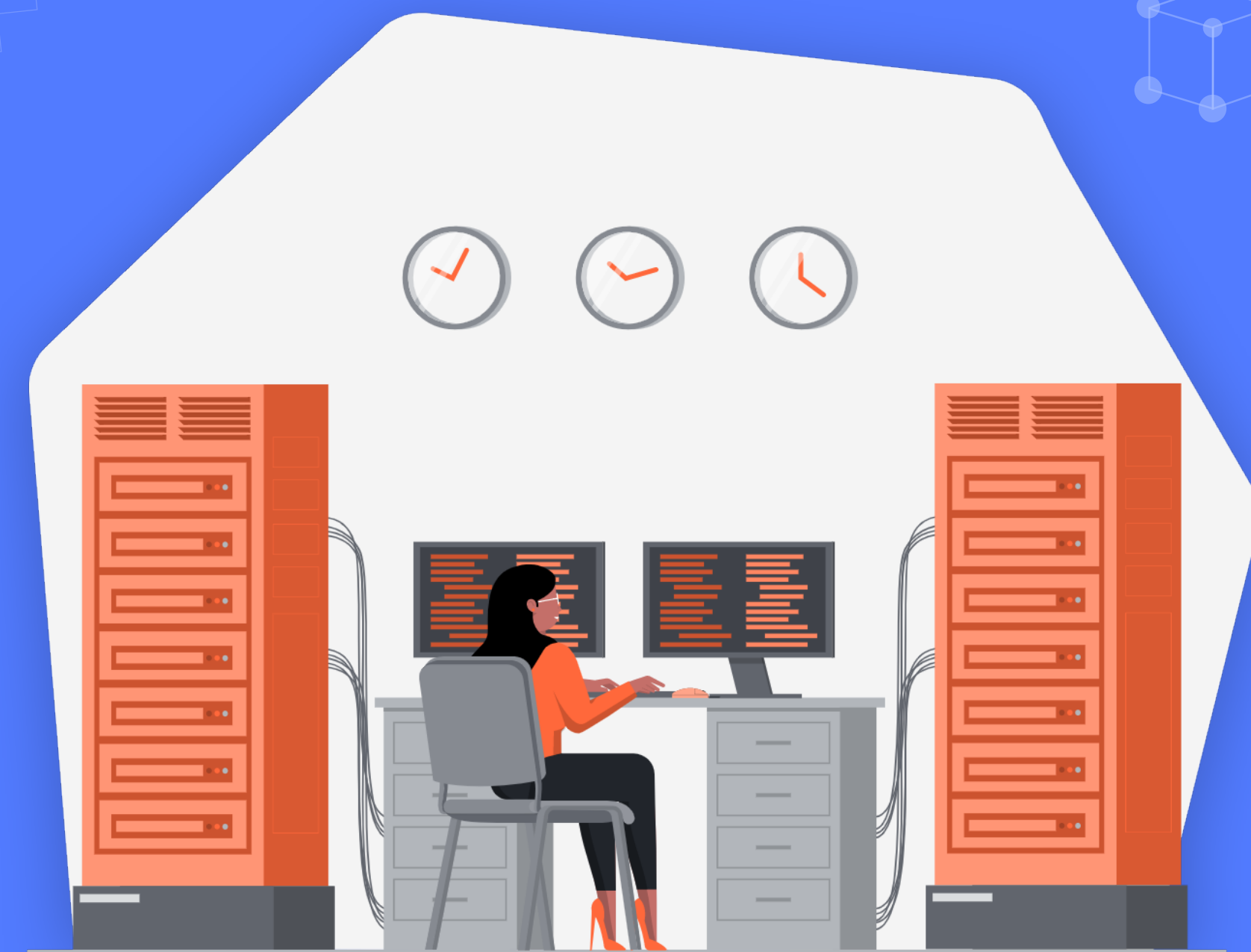


**Patrón
Prototype**

**Patrón
Builder**

**Practica Prototype y
Builder**

Patrón Abstract Factory





Definiciones



Es un patrón de diseño **creacional** que se utiliza para proporcionar una **interfaz** para crear **familias de objetos** sin especificar sus clases concretas.

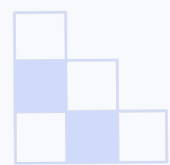


Este patrón se centra en la creación de objetos **relacionados**, también conocidos como familias de objetos. **Proporciona una interfaz abstracta** que declara métodos para la creación de objetos de diferentes tipos.



Ventajas: Patrón Abstract Factory

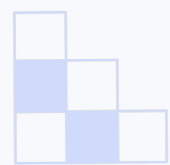
- ✓ **Abstracción de la creación:** separa la creación de objetos de su uso, lo que facilita la sustitución de familias enteras de objetos concretos sin afectar al cliente.
- ✓ **Cumple con el principio de responsabilidad única:** este patrón encapsula la lógica de creación, evitando que el código se entremezcle con la creación de objetos.
- ✓ **Flexibilidad:** Permite la incorporación de nuevas familias de objetos sin modificar el código existente, ya que solo es necesario agregar nuevas clases concretas.





Desventajas: Patrón Abstract Factory

- ✓ **Rigidez en la adición de nuevos tipos:** Agregar nuevos tipos de productos o familias de productos puede requerir cambios en la interfaz y en todas las clases concretas, lo que podría ser costoso los cambios.



Practica del Patrón Abstract Factory



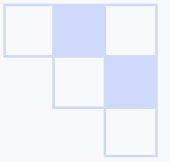
Online-java



<https://www.online-java.com/>

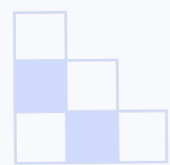
```
</> ONLINE JAVA BETA
Main.java +
1
2 /*
3 Online Java - IDE, Code Editor, Compiler
4
5 Online Java is a quick and easy tool that helps you to build, compile, test your programs online.
6 */
7
8 public class Main
9 {
10     public static void main(String[] args) {
11         System.out.println("Welcome to Online IDE!! Happy Coding :)");
12     }
13 }
14
Ln: 14, Col: 1
```

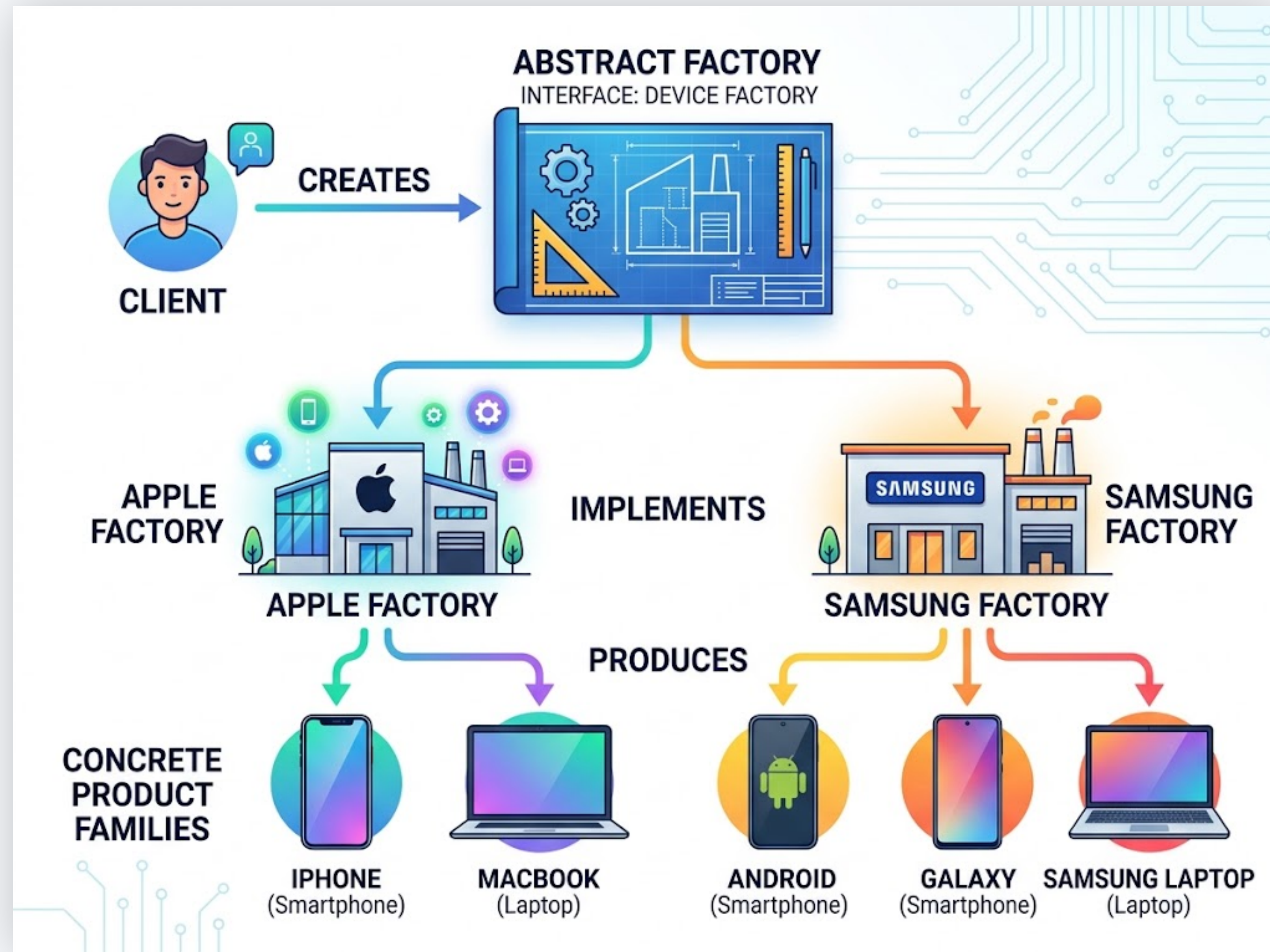




Contexto del caso de estudio

- ✓ Supongamos que tenemos dos tipos de **productos**:
Phone (Teléfono) y **Computer (Computadora)**, y
queremos tener dos marcas de productos:
Samsung y **Apple**.





Código de la clase (Products 1)

```
1  interface Phone {  
2      String getModel();  
3  }  
4  
5  interface Computer {  
6      String getModel();  
7  }
```

Código de la clase (Products 2)

```
9  class SamsungPhone implements Phone {
10      public String getModel() {
11          return "Samsung Galaxy";
12      }
13  }
14
15  class ApplePhone implements Phone {
16      public String getModel() {
17          return "iPhone";
18      }
19  }
20
21  class SamsungComputer implements Computer {
22      public String getModel() {
23          return "Samsung Laptop";
24      }
25  }
26
27  class AppleComputer implements Computer {
28      public String getModel() {
29          return "MacBook Pro";
30      }
31  }
```

Código de la clase (Products 3)

```
33 // Interfaz Abstract Factory
34 interface DeviceFactory {
35     Phone createPhone();
36     Computer createComputer();
37 }
```

Código de la clase (Products 4)

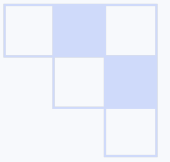
```
39 // Implementation of the abstract factory
40 class SamsungFactory implements DeviceFactory {
41     public Phone createPhone() {
42         return new SamsungPhone();
43     }
44
45     public Computer createComputer() {
46         return new SamsungComputer();
47     }
48 }
49
50 class AppleFactory implements DeviceFactory {
51     public Phone createPhone() {
52         return new ApplePhone();
53     }
54
55     public Computer createComputer() {
56         return new AppleComputer();
57     }
58 }
```

Código de la clase (Products 5)

```
60 public class Main {
61     public static void main(String[] args) {
62         // Create a Samsung device
63         DeviceFactory samsungFactory = new SamsungFactory();
64         Phone samsungPhone = samsungFactory.createPhone();
65         Computer samsungComputer = samsungFactory.createComputer();
66
67         System.out.println("Samsung Phone: " + samsungPhone.getModel());
68         System.out.println("Samsung Computer: " + samsungComputer.getModel());
69
70         // Create a Apple device
71         DeviceFactory appleFactory = new AppleFactory();
72         Phone applePhone = appleFactory.createPhone();
73         Computer appleComputer = appleFactory.createComputer();
74
75         System.out.println("Apple Phone: " + applePhone.getModel());
76         System.out.println("Apple Computer: " + appleComputer.getModel());
77     }
78 }
```

Taller sobre Patrón Abstract Factory





Taller Patrón Abstract Factory aplicado en un caso de estudio real

- ✔ Seleccione un caso de estudio y aplique el patrón de **Abstract Factory**. El script debe ser realizado en **java**.

